

Relazione del primo progetto di Big Data Security by design nei datalake

Il progetto consiste nell'implementare la cybersecurity all'interno del datalake, realizzando una piattaforma big data end-to-end per l'analisi del traffico di rete. Per rendere la simulazione più realistica, tutto ciò è stato implementato su Kathara, che simula una rete grazie alla containerizzazione, usando il software Docker. Eseguendo ciò, ogni dispositivo IS o ES corrisponde ad un container Docker. Perciò, l'architettura big data implementata è situata su un data center di tipo fat-tree.

La rete è divisa in tre AS, dove: AS1 è situata la macchina del presunto hacker, AS2 è dove è situato il data-center e l'admin che gestisce l'architettura e AS3 è situata la risoluzione dei nomi.

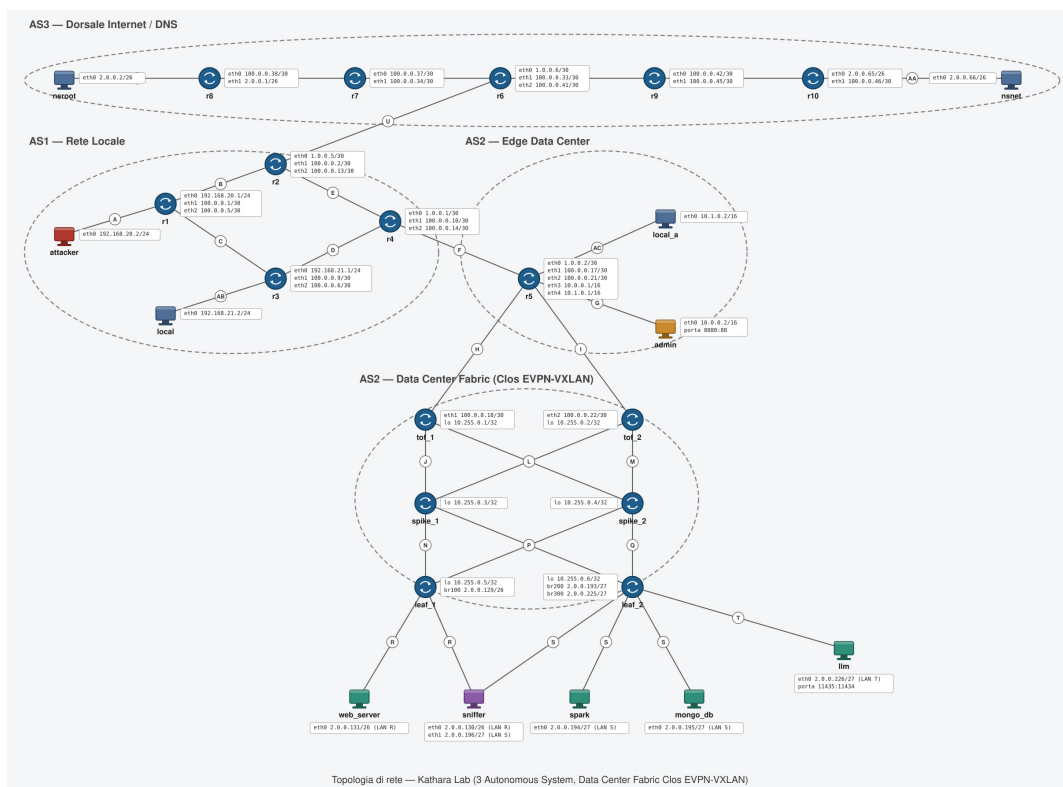


Figura 1: Topologia della rete

Per quanto riguarda, l'architettura, sono stati implementati:

- Spark come motore per l'elaborazione distribuita;
- MongoDB come data-lake operativo;
- LLM locali per l'assistente AI, grazie l'uso di Ollama;
- Streamlit per una dashboard molto semplice da implementare, poiché la parte di HTML, CSS e Javascript è del tutto trasparente al programmatore, garantendo delle pagine Web semplici ed soprattutto intuitive.

1 Dataset di riferimento

Il dataset BigFlow-NIDS, recuperato da Kaggle, contiene flussi di traffico etichettati per il rilevamento delle intrusioni di rete (NIDS). Contiene oltre 66.355.798 record suddivisi tra traffico legittimo (Benign) e diverse categorie di attacco (DDoS, Brute Force, ecc.). Il dataset viene convertito da CSV a Apache Parquet tramite Spark al primo avvio, poiché quest'ultimo garantisce:

- compressione colonnare, che struttura i dati per colonna invece che per riga, riducendo lo spazio occupato e velocizzando la ricerca;
- schema-on-read, cioè un metodo di gestione di dati che salva le informazioni grezze senza una struttura fissa e lo schema viene applicato solo quando i dati vengono letti;
- compatibilità nativa con Spark-SQL.

Di seguito, vengono riportati il codice Python di ingestione del dataset e conversione in Parquet e gli attributi.

```

1  spark = None
2
3  try:
4      spark = SparkSession.builder \
5          .appName("DataLake_Ingestion") \
6          .master("spark://spark-master:7077") \
7          .config("spark.hadoop.fs.permissions.umask-mode", "000") \
8          .config("spark.speculation", "false") \
9          .getOrCreate()
10
11  csv_path = "/opt/spark/data/raw/BigFlow-NIDS.csv"
12  output_path = "/opt/spark/data/processed/BigFlow-NIDS.parquet"
13
14  try:
15      start_time = time.time()
16
17      print(f"Lettura del file CSV {csv_path} ...")
18      df = spark.read.option("header", "true") \
19          .option("inferSchema", "true") \
20          .option("samplingRatio", "0.1") \
21          .csv(csv_path)
22
23      df = df.withColumn("label", df["label"].cast("string"))
24
25      print(f"Scrittura in formato Parquet in corso ...")
26      df.write.mode("overwrite").parquet(output_path)
27
28      end_time = time.time()
29      print(f"Ingestione completata in {round(end_time - start_time, 2)} secondi!")
30
31  except Exception as e:
32      print(f"ERRORE durante l'ingestione: {e}")
33
34  except Exception as e:
35      print("Errore critico di sistema:")
36      print(f"\t{e}")
37
38  finally:
39      if spark is not None:
40          print("Chiusura di spark ...")
41          spark.stop()
42          print("Spark chiuso correttamente")
43

```

Codice 1: Ingestione CSV e conversione in Parquet

Campo	Tipo	Descrizione
IPV4_SRC_ADDR	String	IP sorgente del flusso
IPV4_DST_ADDR	String	IP destinazione del flusso
L4_SRC_PORT / L4_DST_PORT	Integer	Porta sorgente / destinazione
PROTOCOL	Integer	Protocollo (6=TCP, 17=UDP, 1=ICMP)
IN_BYTES / OUT_BYTES	Long	Byte ricevuti / trasmessi
IN_PKTS / OUT_PKTS	Long	Pacchetti ricevuti / trasmessi
FLOW_DURATION_MILLISECONDS	Long	Durata del flusso in ms
FLOW_START_MILLISECONDS	Long	Timestamp di inizio (epoch ms)
label / Attack	String	Tipo di traffico (es. Benigno, DoS)

Tabella 1: Attributi del dataset BigFlow-NIDS

2 Datalake

Come datalake è stato scelto MongoDB, poiché contiene dati live semi-strutturati e documentali. In particolare, offre:

- uno schema flessibile, utilizzando dei documenti a struttura variabile, ideali per PCAP stream ed alert;
- dei bufferi circolari a dimensioni fissa;
- ricerca full-text, grazie all'uso di espressioni regolari;
- connettore con Spark.

2.1 Traffico live

La collezione del traffico live cattura i pacchetti dallo sniffer, inserito nel data center: in particolare è di tipo capped, cioè una struttura circolare in cui i documenti più vecchi vengono sovrascritti con quelli nuovi, poiché i log troppo vecchi diventano ormai obsoleti ed occupano soltanto molta memoria. Nel progetto, la dimensione massima è di solamente 5000 pacchetti, di dimensione pari a 10 MB, inseriti solamente come test: ovviamente, in scenari reali questo limite potrà essere alzato di moltissimo.

Ogni documento presenta la seguente struttura:

- timestamp;
- summary (descrizione tramite scapy);
- length;
- src;
- dst;
- proto.

2.2 Data lake explorer

Il data-lake explorer monta due view federate interrogabili con una singola query SQL:

- file Parquet;
- collezione MongoDB.

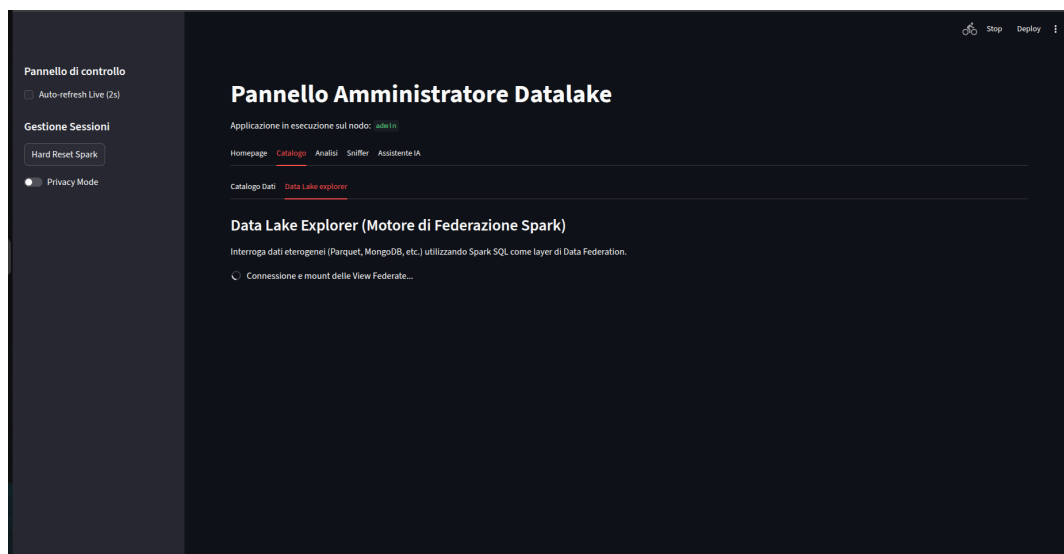


Figura 2: Dashboard del data-lake explorer

A questo punto, l'admin può interrogare il data-lake inserendo delle query-SQL: ovviamente sono permesse solamente delle interrogazioni per motivi di sicurezza, permettendo solamente alcune parole chiave (es. SELECT, UNION, WITH ecc.). Per aiutare l'admin ad eseguire le query, sono stati implementati due LLM, di cui se ne parlerà nel paragrafo apposito.

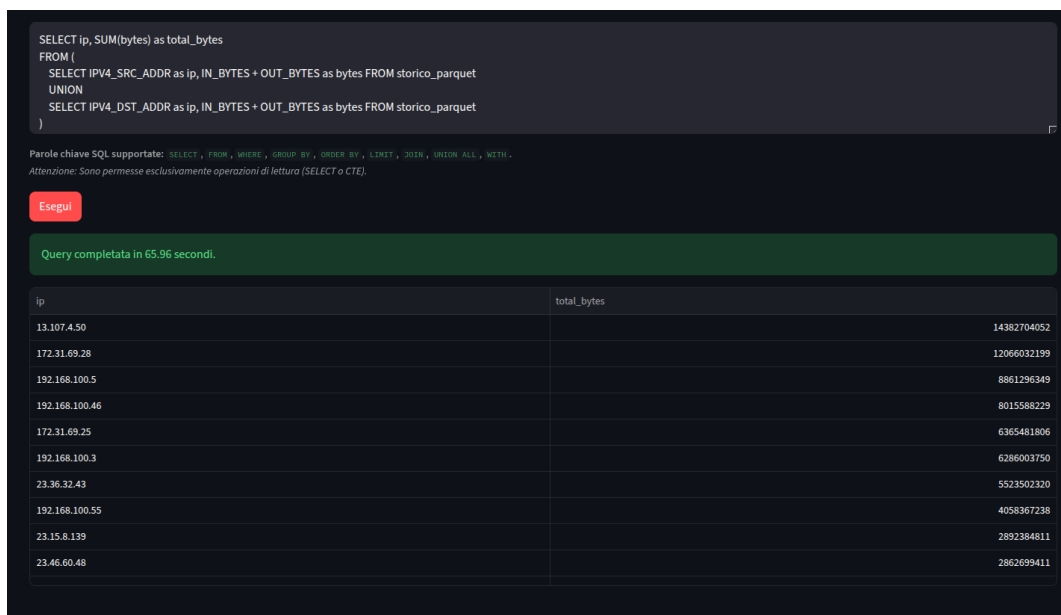


Figura 3: Dashboard dell'esecuzione di una query nel data-lake explorer

3 Analisi del traffico

Tutte le query SQL sono state effettuate con Spark SQL, poiché si hanno oltre 66 milioni di record, perciò è sicuramente la scelta migliore. Invece la scelta di usare Spark SQL invece di Spark core risiede nel fatto che le interrogazioni richiedono delle CTE, raggruppamento, anche case, esprimibili più semplicemente in linguaggio SQL piuttosto che in Python o in Java. Tuttavia, tale scelta è del tutto soggettiva.

3.1 Analisi mitigazione

L'analisi mitigazione consiste nell'eseguire una query che prende i cinque attaccanti con maggior numero di occorrenze, raggruppando per indirizzo IP sorgente e tipologia di traffico malevolo e stampa la quantità di traffico, tutto ciò scritto in formato umano, poiché leggere la quantità solo in byte diventa piuttosto noioso ed inutilmente complicato.

```
1 with traffico as (
2     select ipv4_src_addr as ip_attaccante,
3         case
4             when l4_dst_port = 179 or l4_src_port = 179 then 'BGP Hijacking/
5 Exploit'
6             when l4_dst_port = 22 then 'SSH Brute Force'
7             when l4_dst_port = 53 then 'DNS Amplification'
8             when l4_dst_port = 80 or l4_dst_port = 443 then 'Web Attack (
9 HTTP/S)'
10            when l4_dst_port = 21 then 'FTP File Ingestion Exploit'
11            when flow_duration_milliseconds < 100 and in_pkts > 1000 then '
12 L3/L4 Flood (DDoS)'
13            when in_pkts > 5000 and in_bytes > 100000 then 'Volumetric DDoS'
14            else 'Altro / Malicious General'
15        end as tipo_attacco,
16        count(*) as occorrenze,
17        sum(in_bytes + out_bytes) as traffico_b
18 from traffico_nids
19 where label != 'Benign'
```

```

17     group by ipv4_src_addr, 2
18 )
19
20 select ip_attaccante,
21        occorrenze,
22        tipo_attacco,
23        case
24            when traffico_b >= 1024.0 * 1024.0 * 1024.0 * 1024.0 then concat(
25 round(traffico_b * 1.0 / (1024.0 * 1024.0 * 1024.0 * 1024.0), 2), 'TB')
26            when traffico_b >= 1024.0 * 1024.0 * 1024.0 then concat(round(
27 traffico_b * 1.0 / (1024.0 * 1024.0 * 1024.0), 2), 'GB')
28            when traffico_b >= 1024.0 * 1024.0 then concat(round(traffico_b *
29 1.0 / (1024.0 * 1024.0), 2), 'MB')
30            when traffico_b >= 1024.0 then concat(round(traffico_b * 1.0 /
31 1024.0, 2), 'KB')
32            else concat(traffico_b, 'B')
33        end as traffico_h
34 from traffico
35 order by occorrenze desc
36 limit 5;

```

Interrogazione 2: Analisi del traffico malevolo per indirizzo IP

Come è possibile vedere nella dashboard, è possibile bloccare tali indirizzi IP, inserendoli nella black-list nella tabella di instradamento del router che si interfaccia nel data center.

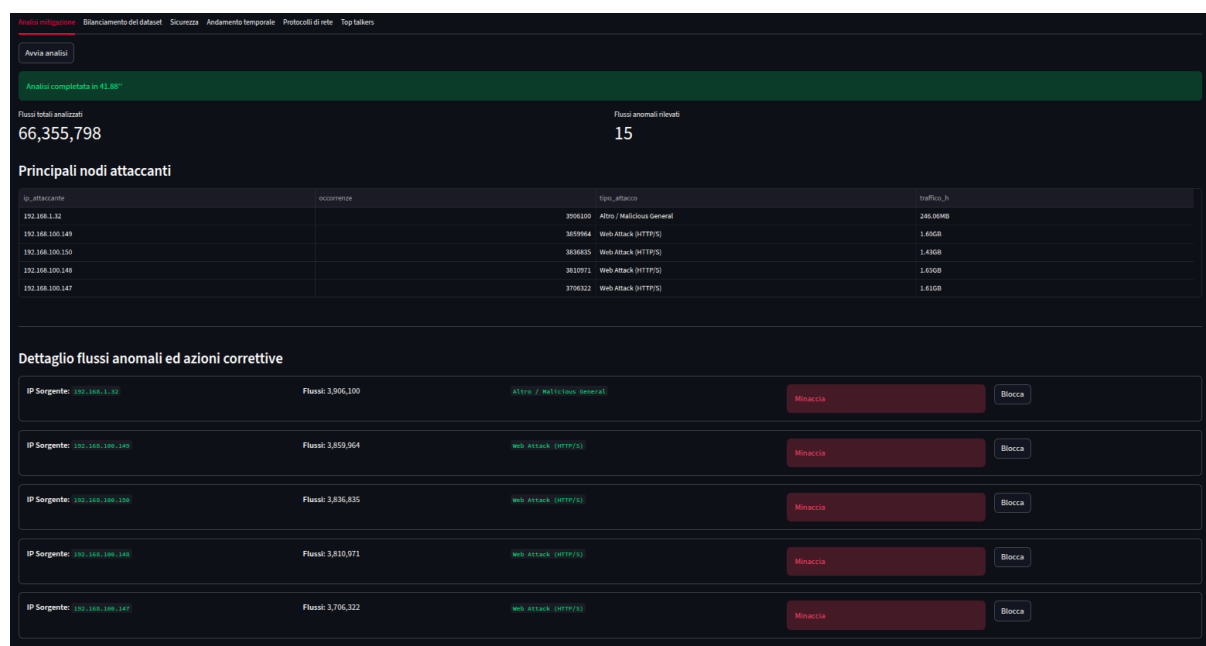


Figura 4: Dashboard che riporta l'analisi mitigazione

In questa query viene eseguita la cosiddetta analisi delle performance, in cui viene confrontato la scalabilità in Spark e con un DB relazionale. In particolare, il tempo del DB relazionale non è misurato, ma stimato: si parte dal tempo reale dell'analisi Spark sui 66M di record. In particolare:

- la funzione Spark si ottiene scalando linearmente il tempo per i tre volumi ($6.6M \rightarrow t * 0.1$, $33M \rightarrow t * 0.5$, $66M \rightarrow t$);
- la funzione "Legacy DB (Projected)" usa invece moltiplicatori fissi sullo stesso tempo (1.5x, 7x, 15x).

Questi fattori simulano un degrado superlineare tipico di un DB relazionale singolo che perde efficienza all'aumentare del volume, eseguendo quindi solo una proiezione euristica codificata a mano. Ovviamente, il confronto serve solamente a scopo illustrativo per mostrare il vantaggio atteso di Spark su grandi volumi. Tutto ciò viene mostrato nella figura seguente.

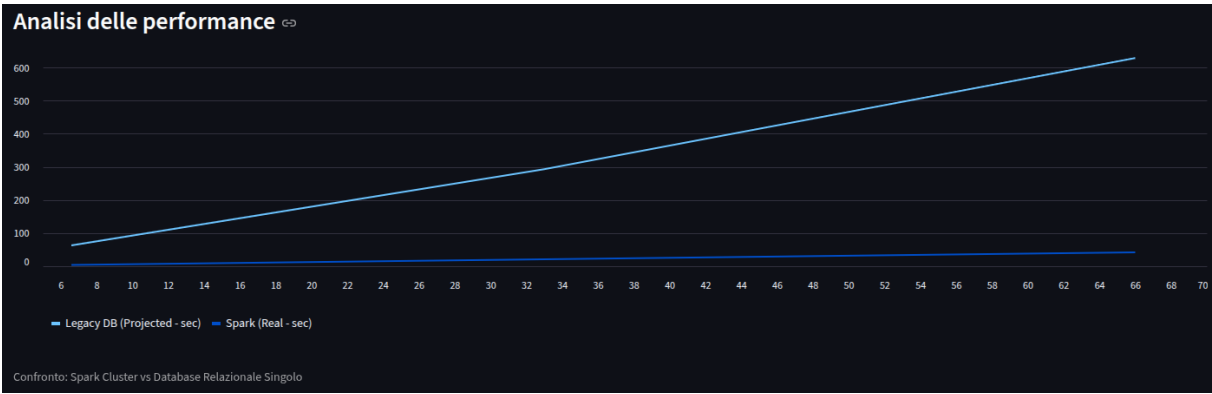


Figura 5: Grafico delle performance tra Spark e DB relazionale

3.2 Bilanciamento del dataset

La seconda query riguarda il bilanciamento del dataset, che riporta in percentuale come è diviso il traffico. Ciò è fondamentale per riconoscere se il tipo di traffico verso il data lake è normale traffico oppure è un attacco. Sicuramente avere un dataset fortemente sbilanciato (come nel dataset di riferimento) non aiuta, poiché il modello si troverà in difficoltà nel riconoscere gli attacchi. Inoltre, ciò risulterà utile se si vorrà implementare in futuro modelli di Machine Learning usando la libreria di Spark MLlib.

```
1 select Attack as label, count(*) as occorrenze
2 from traffico_nids
3 group by Attack
4
```

Interrogazione 3: Bilanciamento del dataset

Nella dashboard ciò viene rappresentato con un grafico a barre e con una tabella delle frequenze.

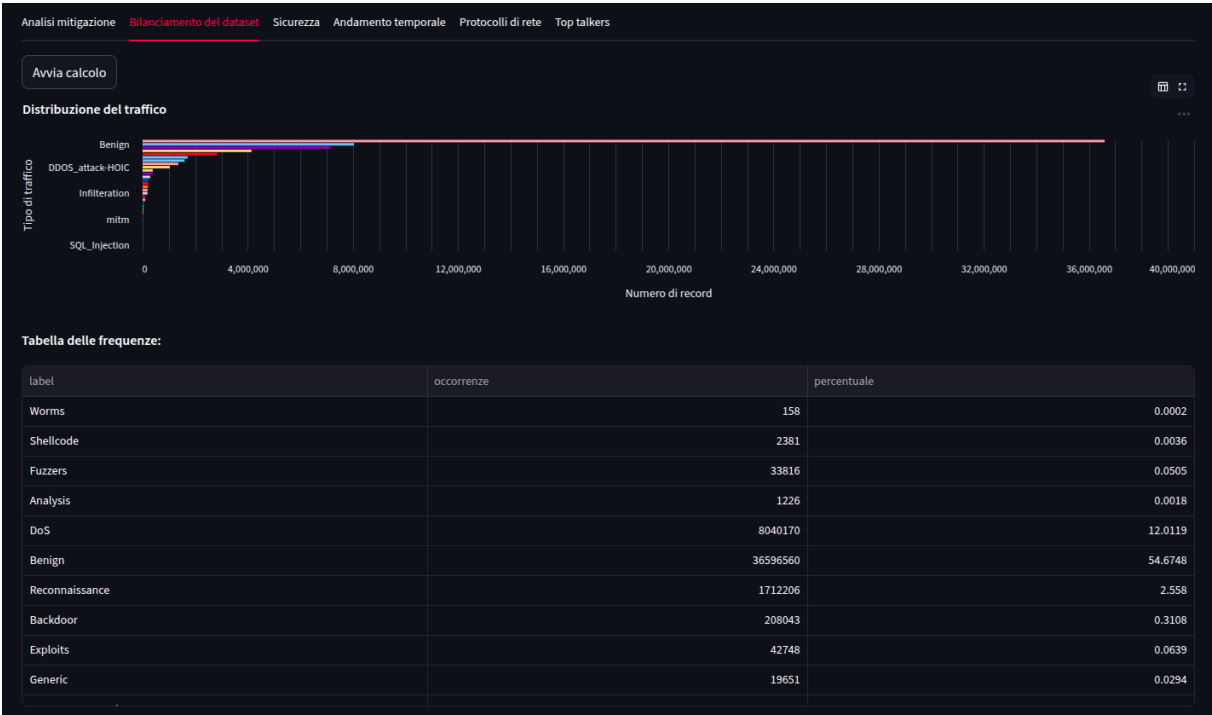


Figura 6: Dashboard che riporta il bilanciamento del dataset

3.3 Sicurezza

Per quanto riguarda la sicurezza, il sistema implementa un ciclo di rilevamento sfruttando lo sniffer che cattura i pacchetti, esegue le query del rilevamento delle anomalie e avvisa immediatamente l'utente se il data center è sotto attacco. In quel caso, l'amministratore può bloccare immediatamente il traffico, bloccando l'IP dell'attaccante con un messaggio di "Destination Port Unreachable". Ciò avviene perché nel router di ingresso del data center gira un demone in Python che legge da MongoDB tutti gli IP che hanno uno status di bloccato e rispedisce le regole sulle sue tabelle di instradamento al router.

Tutto ciò viene riportato dalle figure che seguono.

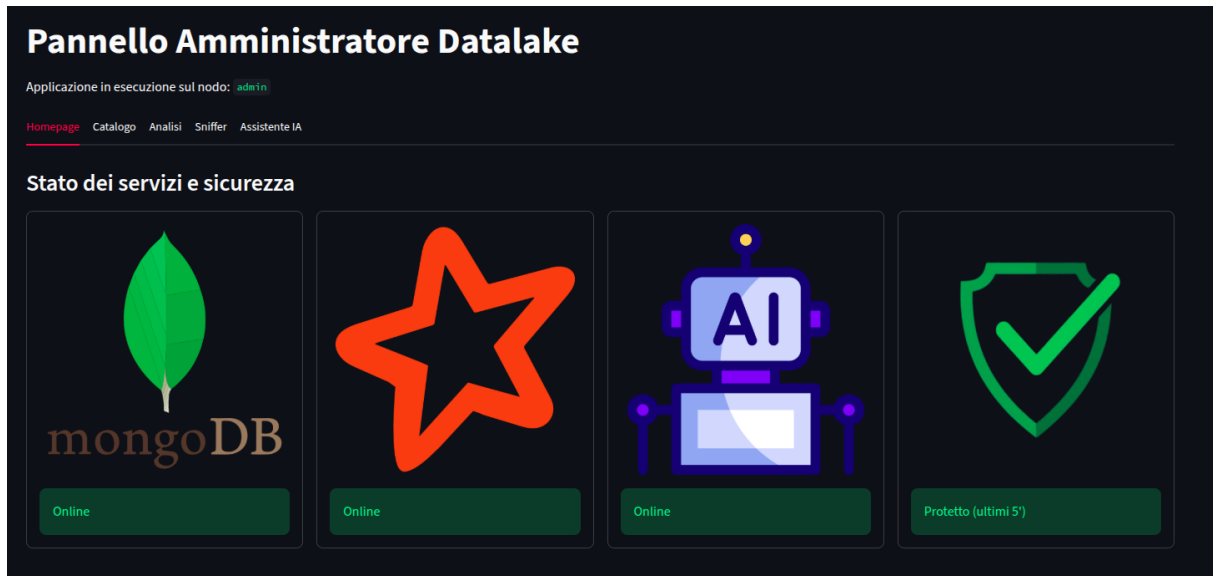


Figura 7: Homepage della dashboard con tutto funzionante

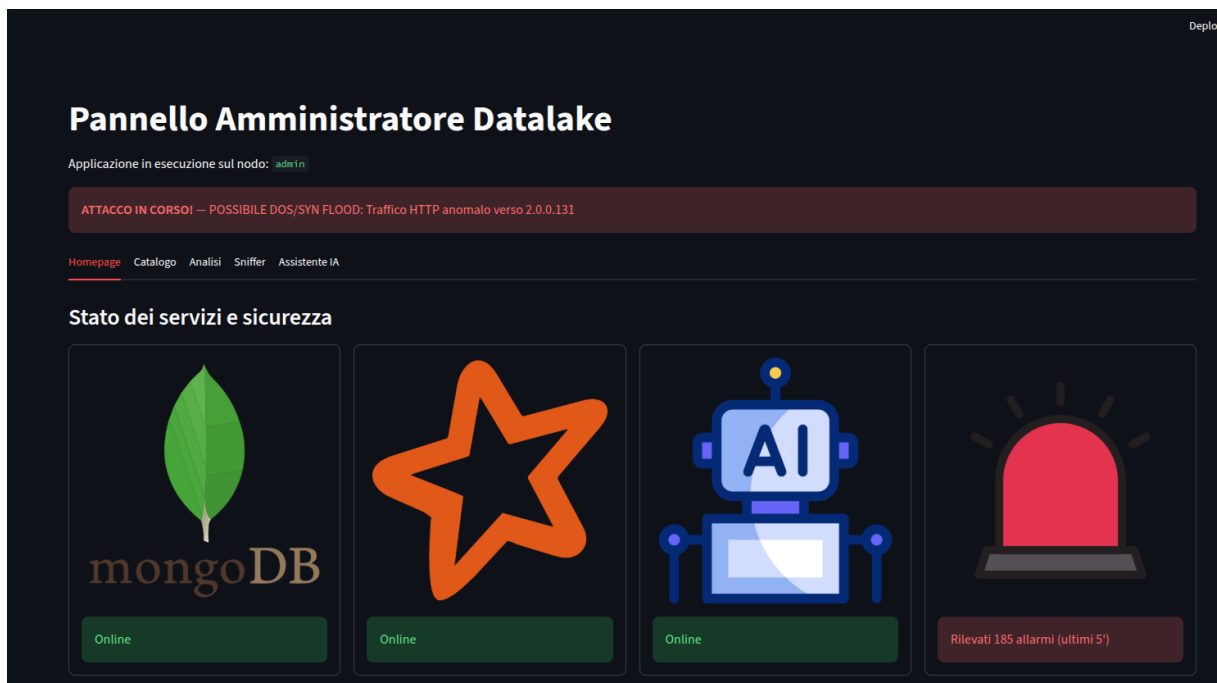


Figura 8: Homepage della dashboard quando viene rilevato un attacco

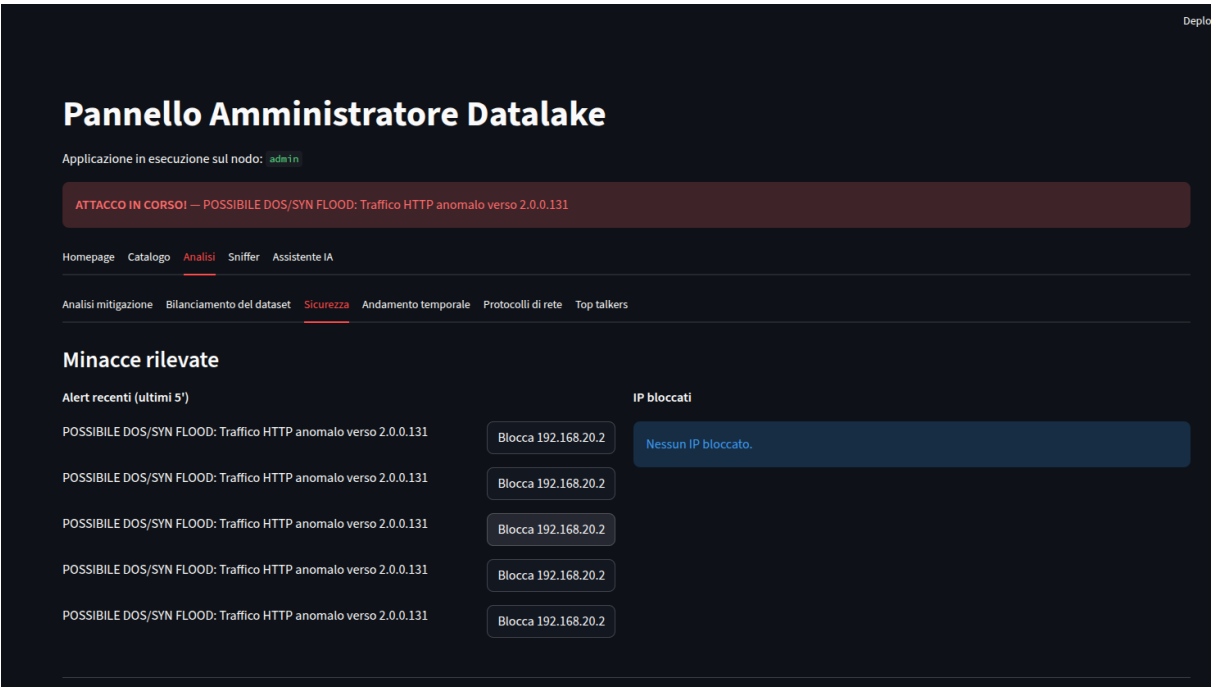


Figura 9: Dashboard per bloccare l'IP di un presunto attacco

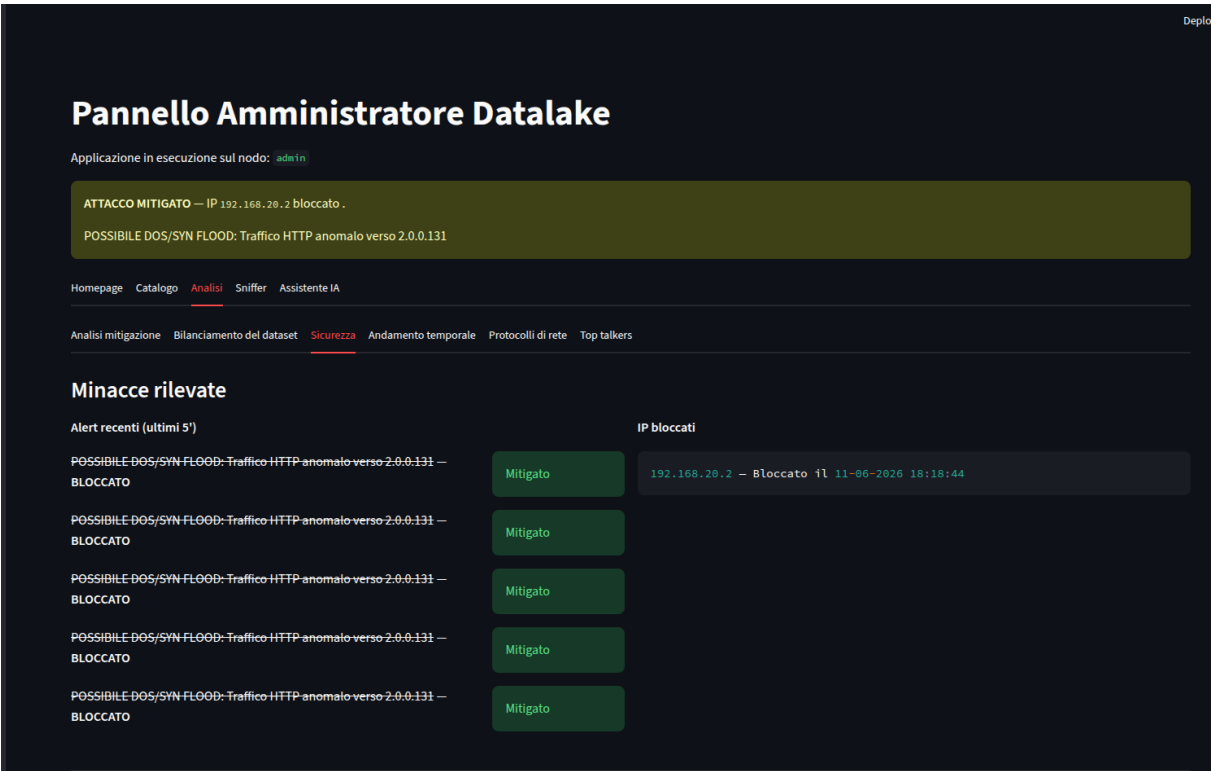


Figura 10: Dashboard per che mostra gli IP bloccati


```

root@attacker:/# hping3 -S -V -p 80 mongo.cyber.net
using eth0, addr: 192.168.20.2, MTU: 1500
HPING mongo.cyber.net (eth0 2.0.0.195): S set, 40 headers + 0 data bytes
len=46 ip=2.0.0.195 ttl=57 DF id=0 tos=0 iplen=40
sport=80 flags=RA seq=0 win=0 rtt=5.6 ms
seq=0 ack=755652639 sum=a147 urp=0

len=46 ip=2.0.0.195 ttl=57 DF id=0 tos=0 iplen=40
sport=80 flags=RA seq=1 win=0 rtt=5.3 ms
seq=0 ack=1305843829 sum=7ee6 urp=0

len=46 ip=2.0.0.195 ttl=57 DF id=0 tos=0 iplen=40
sport=80 flags=RA seq=2 win=0 rtt=6.1 ms
seq=0 ack=756595461 sum=3c44 urp=0

len=46 ip=2.0.0.195 ttl=57 DF id=0 tos=0 iplen=40
sport=80 flags=RA seq=3 win=0 rtt=5.9 ms
seq=0 ack=1035501268 sum=3b10 urp=0

^C
--- mongo.cyber.net hping statistic ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 5.3/5.7/6.1 ms
root@attacker:/#

```

Figura 11: Terminale dell'attaccante che sta eseguendo l'attacco

```

root@attacker:/# ping -c 1 www.cyber.net
PING www.cyber.net (2.0.0.131) 56(84) bytes of data:
From 1.0.0.2 (1.0.0.2) icmp_seq=1 Destination Port Unreachable

--- www.cyber.net ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

root@attacker:/#

```

Figura 12: Terminale dell'attaccante che è stato bloccato

3.4 Analisi dell'andamento temporale

L'analisi dell'andamento temporale permette di aggregare i flussi di traffico in un minuto, per calcolare l'andamento del traffico sia benigno sia malevolo, e di identificare i picchi e le finestre di attività malevola, riportando un grafico multilinee.

```

1  select
2      date_trunc('minute', from_unixtime(flow_start_milliseconds / 1000)) as
time_window,
3      label, sum(in_bytes + out_bytes) as total_bytes, count(*) as flow_count
4  from traffico_nids
5  group by 1, 2
6  order by 1 asc
7

```

Interrogazione 4: Andamento temporale

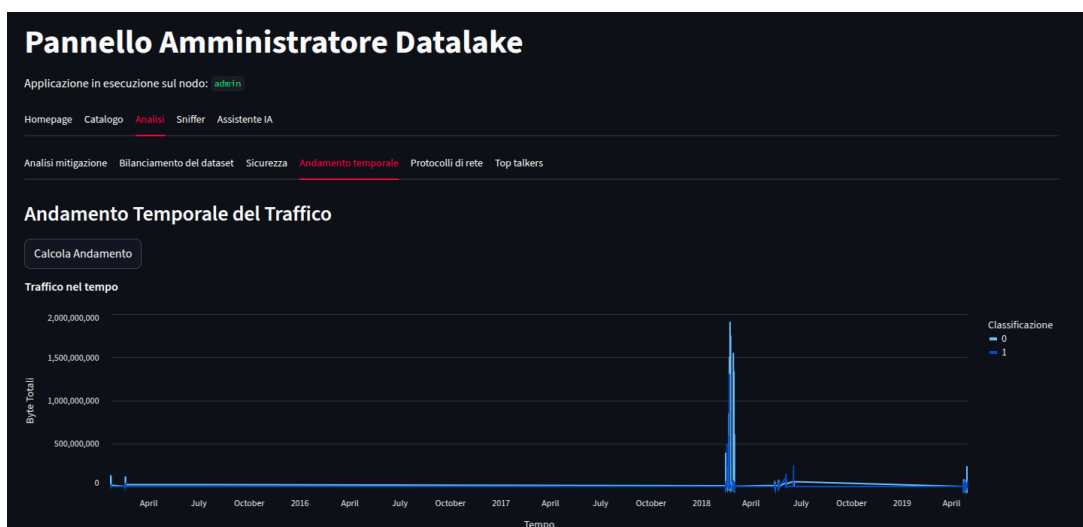


Figura 13: Grafico multilinee che mostra l'andamento temporale del traffico

3.5 Distribuzione dei protocolli di rete e flussi

Un'altra interrogazione molto importante è la distribuzione dei protocolli di rete e delle porte utilizzate durante gli attacchi, mostrando quali sono le porte più critiche. Ciò viene mostrato da un grafico a barre a pila, distinguendo da traffico benigno a traffico malevolo.

```

1      select
2          l4_dst_port as port,
3          case
4              when protocol = 6 then 'TCP'
5              when protocol = 17 then 'UDP'
6              when protocol = 1 then 'ICMP'
7              else cast(protocol as string)
8          end as protocol_name,
9          label,
10         count(*) as flow_count,
11         avg(in_bytes + out_bytes) as avg_bytes
12 from traffico_nids
13 group by 1, 2, 3
14 order by flow_count desc
15 limit 50
16

```

Interrogazione 5: Distribuzione dei protocolli e flussi

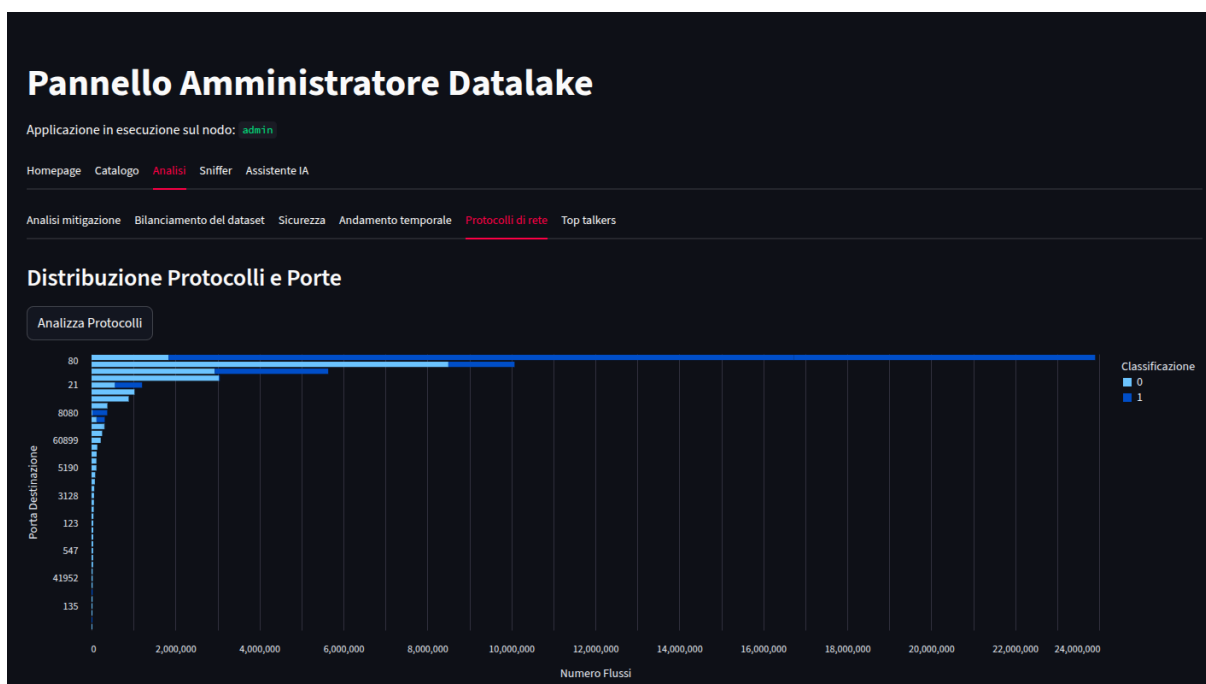


Figura 14: Grafico a barre a pila che mostra la distribuzione dei protocolli e flussi

3.6 Top talkers

L'ultima query si basa nel creare una heatmap tra IP sorgente ed IP destinazione, per rappresentare il volume dei byte. Ciò è molto efficace per evidenziare gli attacchi volumetrici causati da una singola sorgente IP.

```

1      select
2          ipv4_src_addr as source_ip,
3          ipv4_dst_addr as destination_ip,
4          label,
5          sum(in_bytes + out_bytes) as total_bytes,
6          count(*) as flow_count
7 from traffico_nids

```

```
8 where ipv4_src_addr is not null and ipv4_dst_addr is not null
9 group by 1, 2, 3
10 order by total_bytes desc
11 limit 100
12
```

Interrogazione 6: Top talkers

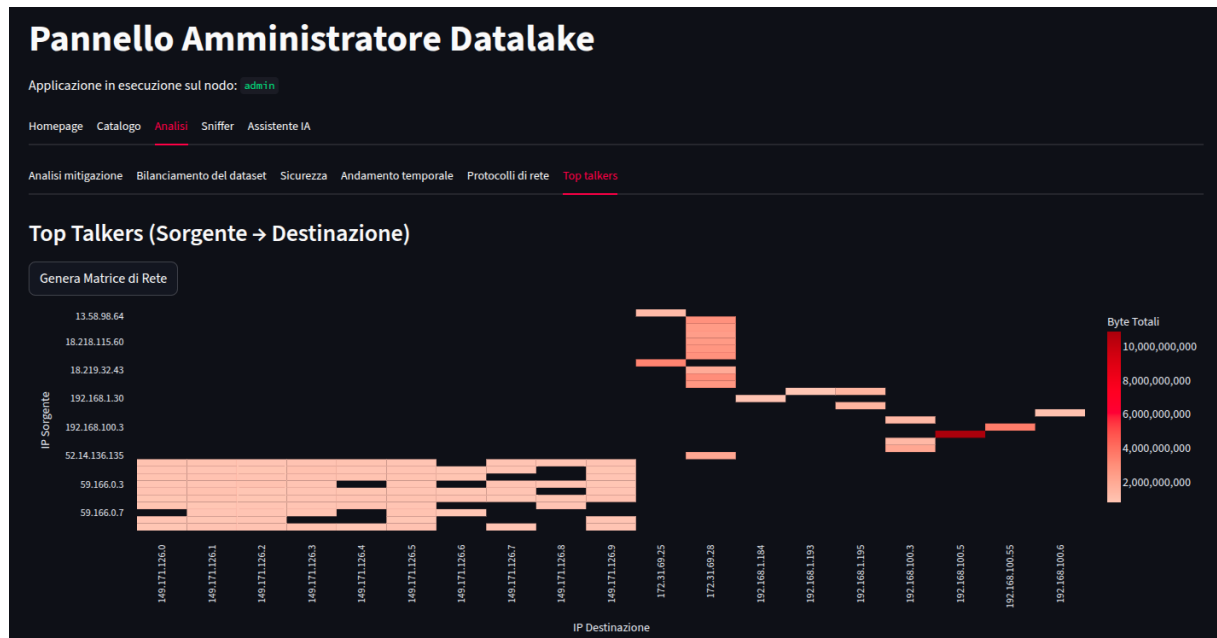


Figura 15: Heatmap che mostra quali sono gli IP sorgenti che inviano più volumi di dati

4 Implementazione degli LLM

In questo progetto sono stati implementati due LLM in locale, sfruttando il motore di inferenza Ollama. La scelta degli LLM si basano molto sulle caratteristiche dell'hardware del calcolatore usato, mostrate di seguito.

```
Specifiche di Esecuzione (Ambiente Locale)
$ lab_release -d; lscpu; free -h; sudo lshw; java -version

===== SISTEMA OPERATIVO =====
Description: Ubuntu 24.04.4 LTS

===== CPU E CORE =====
CPU(s): 18
Model name: Intel(R) Core(TM) i7-240H
Thread(s) per core: 2
Core(s) per socket: 10

===== MEMORIA RAM =====
              total        used        free      shared  buff/cache   disponibile
Mem:          15Gi          8.5Gi          3.1Gi          1.8Gi          6.1Gi          6.8Gi

===== SCHEDA VIDEO =====
riporto: iomemory:600-5ff iomemory:620-61f irq:10 memoria:84000000-07ffffff memoria:600000000-61ffffff memoria:620000000-6201ffffff ioport:5000(dimensione=128) memoria:00000000-0007ffff
riporto: iomemory:600-61f iomemory:600-3ff irq:102 memoria:6003000000-6027ffffff memoria:4000000000-4007ffffff ioport:6000(dimensione=64) memoria:c0000-0ffff memoria:4030000000-4036ffffff memoria:4020000000-4027ffffff

===== AMBIENTE SOFTWARE =====
openjdk version "11.0.36" 2020-01-20
```

Figura 16: Specifiche del calcolatore utilizzato per il progetto

Infatti, siccome i parametri degli LLM sono parecchi, il carico è stato diviso tra RAM (16 GB) e VRAM (8 GB), per evitare rallentamenti e soprattutto crash per out of memory.

Inoltre, è stato scelto di utilizzare MongoDB come contesto operativo, invece che un VectorDB, perché il recupero dei dati è documentale (es. report storici e documentazione), compito in cui MongoDB riesce ad eseguire tranquillamente. Nel caso in cui il retrieval fosse di tipo vettoriale o non deterministico, allora la complicazione architetturale avrebbe avuto sicuramente più senso.

Per quanto riguarda la scelta degli LLM, sono stati implementati:

- llama 3.2 (indicato con "Veloce"), con 3 miliardi di parametri, per fornire risposte rapide;
- qwen 3.5 (indicato con "Ragionamento"), con 9 miliardi di parametri, per fornire risposte ragionate, che hanno i tag <think> espliciti.

La comunicazione con Ollama avviene tramite API in modalità streaming, separando i token di ragionamento da quelli della risposta finale tramite espressione regolare.

Algoritmo 1 Streaming della risposta di un LLM con separazione tra ragionamento e risposta finale

```

1: procedure STREAMLLMRESPONSE(model_id, prompt, system_instructions, thinking_area, answer_area, timer_area, format_sql,
  llm_options)
2:   inizio ← TEMPOCORRENTE
3:   durata_pensiero ← 0
4:   testo_pensiero ← " "
5:   risposta_pulita ← " "
6:   flusso_grezzo ← " "
7:   if "3b" ∈ MINUSCOLO(model_id) then
8:     system_instructions ← system_instructions + istruzione_extra
9:   end if
10:  opzioni ← {temperature : 0.3, top_p : 0.85, num_predict : 2048}
11:  if llm_options fornito then
12:    opzioni ← opzioni ∪ llm_options
13:  end if
14:  try
15:    risposta ← RICHIESTAHTTP_POST(server_LLM, model_id, prompt,
16:      system_instructions, opzioni, stream = vero)
17:  if risposta.stato = 200 then
18:    for each riga in risposta (streaming) do
19:      if riga non vuota then
20:        chunk ← PARSEJSON(riga)
21:        token_risposta ← chunk.response
22:        token_pensiero ← chunk.thinking
23:        trascorso ← TEMPOCORRENTE − inizio
24:        if token_pensiero non vuoto then
25:          durata_pensiero ← trascorso
26:          testo_pensiero ← testo_pensiero + token_pensiero
27:          risposta_pulita ← risposta_pulita + token_risposta
28:        else
29:          flusso_grezzo ← flusso_grezzo + token_risposta
30:          if "<think>" ∈ flusso_grezzo then
31:            durata_pensiero ← trascorso
32:            testo_pensiero ←
33:              ESTRAITRATAG(flusso_grezzo, <think>, </think>)
34:            risposta_pulita ←
35:              RIMUOVITAG(flusso_grezzo, <think>...</think>)
36:          else
37:            risposta_pulita ← flusso_grezzo
38:          end if
39:        end if
40:        if timer_area definito then
41:          aggiorna timer_area con trascorso
42:        end if
43:        if testo_pensiero non vuoto then
44:          aggiorna thinking_area con testo_pensiero e durata_pensiero
45:        end if
46:        if risposta_pulita non vuota then
47:          if format_sql then
48:            mostra risposta_pulita in answer_area come codice SQL
49:          else
50:            mostra risposta_pulita in answer_area come markdown
51:          end if
52:        end if
53:      end if
54:    end for
55:    return (risposta_pulita, testo_pensiero,
56:      TEMPOCORRENTE − inizio, durata_pensiero)
57:  else
58:    mostra errore("Stato " + risposta.stato)
59:    return (null, null, 0, 0)
60:  end if
61:  catch
62:    mostra errore(eccezione)
63:    return (null, null, 0, 0)
64: end procedure

```

4.1 Assistente IA

L'assistente IA implementa un RAG contestuale in cui viene implementata una forma molto semplificata di Retrieval-Augmented Generation, che consiste nel recuperare dal data lake, in questo caso MongoDB, i 5 IP bloccati ed i 5 alert più recenti e li inietta nel prompt come contesto reale.

Algoritmo 2 RAG contestuale per l'AI Copilot

```
1: kw ← ["ip", "bloccat", "firewall", "traffico", ...]
2: greetings ← ["ciao", "hello", "chi sei", ...]
3: generic ← (len(prompt) ≤ 15 or prompt ∈ greetings)
4: if generic then
5:   prompt_finale ← "Rispondi in italiano: " + prompt
6: else
7:   blocked ← MONGODB.BLOCKED_IPS.FIND({"status": "BLOCKED"}, limit = 5)
8:   alerts ← MONGODB.ALERTS.FIND({}, sort = {ts: -1}, limit = 5)
9:   contesto ← FORMATTA(blocked, alerts)
10:  prompt_finale ← contesto + "\n Domanda: " + prompt
11: end if
12: return OLLAMASTREAM(model, prompt_finale, system)
```

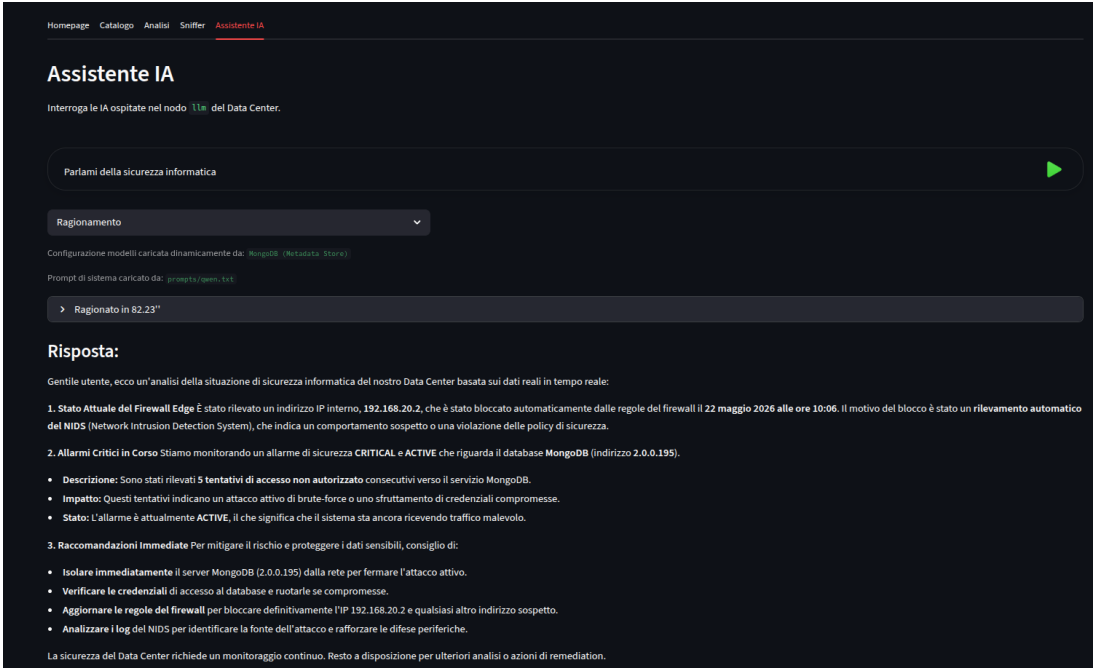


Figura 17: Assitente IA in modalità ragionamento

```
1 Sei un assistente virtuale esperto di cybersecurity per il Data Center.
2 IMPORTANTE: Devi rispondere SEMPRE in lingua ITALIANA in modo diretto, veloce e
  conciso.
3
4 Linee guida per la tua risposta:
5 1. Sii chiaro, professionale e usa sempre un italiano corretto. Non inserire monologhi
  interni o fasi di ragionamento.
6 2. Usa termini tecnici italiani appropriati (es. "regole del firewall", "traffico
  bloccato", "indirizzi IP").
7 3. Assisti l'utente con l'analisi del traffico, gli IP bloccati e gli allarmi del
  firewall in modo cortese ed efficiente.
8
9
```

Interrogazione 7: Invocazione di llama

Sei un assistente virtuale esperto di cybersecurity per il Data Center.
 IMPORTANTE: Devi rispondere SEMPRE **in** lingua ITALIANA.

Linee guida per la tua risposta:

1. Devi sempre eseguire una fase di ragionamento preliminare inserendo i tuoi pensieri tra i tag <think> e </think>.
2. Pensa **in** italiano **all'interno** dei tag <think>.
3. Dopo i tag </think>, scrivi la tua risposta finale, che deve essere chiara, professionale e concisa, sempre in italiano corretto.
4. Usa termini tecnici italiani appropriati (es. "regole del firewall", "traffico bloccato", "indirizzi IP").
5. Assisti l'utente con l'analisi del traffico, gli IP bloccati e gli allarmi del firewall in modo cortese.

Interrogazione 8: Invocazione di qwen

4.2 Generazione SQL via LLM

Come già accennato, nel data lake explorer sono stato implementati llama e qwen per aiutare l'amministratore ad scrivere le query SQL. Ciò avviene riportando solamente le parole chiave concesse per motivi di sicurezza e le tabelle e gli attributi che possiedono tali tabelle. La risposta viene generata in sintassi SQL.

Mostrami gli IP che hanno mostrato più scambi di byte

Ragionamento

Ragionamento in corso: 139.56"

L'utente vuole vedere gli IP con più scambi di byte. Devo decidere quale tabella usare. 'storico_parquet' ha IN_BYTES e OUT_BYTES, mentre 'live_mongo' ha solo summary. Probabilmente devo usare 'storico_parquet'. Gli IP sono IPV4_SRC_ADDR e IPV4_DST_ADDR. Devo sommare IN_BYTES e OUT_BYTES per ogni IP, sia come sorgente che come destinazione. Quindi devo raggruppare per IPV4_SRC_ADDR e IPV4_DST_ADDR separatamente, oppure creare una colonna unificata. Forse è meglio creare una vista temporanea che unisca le colonne di IP. Oppure fare due query separate e unirle. Ma l'utente vuole "gli IP", quindi probabilmente tutti gli IP coinvolti. Quindi posso fare: SELECT IPV4_SRC_ADDR as ip, IN_BYTES + OUT_BYTES as total_bytes FROM storico_parquet UNION SELECT IPV4_DST_ADDR as ip, IN_BYTES + OUT_BYTES as total_bytes FROM storico_parquet GROUP BY ip, total_bytes ORDER BY total_bytes DESC LIMIT 10. Ma aspetta, devo raggruppare per IP. Quindi: SELECT IPV4_SRC_ADDR as ip, SUM(IN_BYTES + OUT_BYTES) as total_bytes FROM storico_parquet GROUP BY IPV4_SRC_ADDR UNION SELECT IPV4_DST_ADDR as ip, SUM(IN_BYTES + OUT_BYTES) as total_bytes FROM storico_parquet GROUP BY IPV4_DST_ADDR ORDER BY total_bytes DESC. Ma forse è meglio usare una CTE per unire le colonne. Oppure: WITH all_ips AS (SELECT IPV4_SRC_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet UNION SELECT IPV4_DST_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet) SELECT ip, SUM(bytes) as total_bytes FROM all_ips GROUP BY ip ORDER BY total_bytes DESC LIMIT 10. Sì, questo sembra corretto. Ma l'utente non ha specificato un limite, quindi forse non serve LIMIT. Ma "più scambi" implica top N. Forse è meglio senza LIMIT. Ma in genere si usa LIMIT 10 o 100. Tuttavia, l'utente non ha specificato, quindi forse è meglio senza. Ma "più scambi" suggerisce un ordinamento. Quindi ORDER BY total_bytes DESC. Quindi la query finale: SELECT ip, SUM(bytes) as total_bytes FROM (SELECT IPV4_SRC_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet UNION SELECT IPV4_DST_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet) GROUP BY ip ORDER BY total_bytes DESC. Ma aspetta, devo usare solo le tabelle 'storico_parquet' e 'live_mongo'. 'live_mongo' ha timestamp, src, dst, proto, summary. Forse 'summary' contiene i byte? Ma non è specificato. Quindi meglio usare 'storico_parquet'. Quindi la query sopra è corretta. Ma forse è meglio usare 'live_mongo' per i dati in tempo reale? Ma l'utente non ha specificato. Quindi uso 'storico_parquet'. Quindi la query è: SELECT ip, SUM(bytes) as total_bytes FROM (SELECT IPV4_SRC_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet UNION SELECT IPV4_DST_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet) GROUP BY ip ORDER BY total_bytes DESC. Ma aspetta, in Spark SQL, UNION allinea le colonne. Quindi è corretto. Quindi la query finale.

```

SELECT ip, SUM(bytes) as total_bytes
FROM (
  SELECT IPV4_SRC_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet
  UNION
  SELECT IPV4_DST_ADDR as ip, IN_BYTES + OUT_BYTES as bytes FROM storico_parquet
)
GROUP BY ip
ORDER BY total_bytes DESC

```

Figura 18: Query generata dal LLM mostrando anche il ragionamento

Sei un assistente esperto **in** Data Engineering, specializzato nella scrittura di query SQL per Apache Spark.

Il tuo compito è scrivere la query SQL esatta richiesta dall'utente in modo rapido e diretto.

Regole fondamentali:

1. Restituisci SOLO ed ESCLUSIVAMENTE il codice SQL. Niente saluti, niente testo extra, niente spiegazioni, niente markdown se non strettamente necessario.
2. La query DEVE iniziare con "SELECT" o "WITH".
3. Utilizza solo le tabelle: 'storico_parquet' e 'live_mongo'.
4. 'storico_parquet' ha le colonne: IPV4_SRC_ADDR, IPV4_DST_ADDR, PROTOCOL, IN_BYTES, OUT_BYTES, label, Attack, FLOW_START_MILLISECONDS.

```
5. 'live_mongo' ha le colonne: timestamp, src, dst, proto, summary.
Non effettuare fasi di ragionamento. Fornisci immediatamente la query richiesta senza
tag extra.
```

Interrogazione 9: Invocazione di llama per generare la query

```
Sei un assistente esperto in Data Engineering, specializzato nella scrittura di query
SQL per Apache Spark.
Il tuo compito è scrivere la query SQL esatta richiesta dall'utente.
Regole fondamentali:
1. Restituisci SOLO ed ESCLUSIVAMENTE il codice SQL. Niente saluti, niente testo extra
, niente spiegazioni, niente markdown se non strettamente necessario.
2. La query DEVE iniziare con "SELECT" o "WITH".
3. Utilizza solo le tabelle: 'storico_parquet' e 'live_mongo'.
4. 'storico_parquet' ha le colonne: IPV4_SRC_ADDR, IPV4_DST_ADDR, PROTOCOL, IN_BYTES,
OUT_BYTES, label, Attack, FLOW_START_MILLISECONDS.
5. 'live_mongo' ha le colonne: timestamp, src, dst, proto, summary.
Usa i tag <think> e </think> per i tuoi ragionamenti interni, ma il blocco finale deve
essere solo codice SQL.
```

Interrogazione 10: Invocazione di qwen per generare la query

5 Ulteriori strumenti

Per la gestione del data lake, sono stati implementati anche altri due strumenti che sono molto utili e rilevanti.

5.1 Catalogo

Il catalogo implementa un metadata store centralizzato su MongoDBmche censisce tutte le fonti dati del sistema (dataset storico NIDS, traffico live, modelli LLM), mostrando per ciascuna schema, stato online/offline, formato e un'anteprima con profilazione automatica delle colonne (tipi di dato, statistiche descrittive). Supporta inoltre l'upload di nuovi dataset CSV con una pipeline di security by design: limite dimensione, controllo estensione, rilevamento di file binari mascherati da CSV tramite magic number, scansione di keyword pericolose (script/comandi) e sanitizzazione di CSV/formula injection.

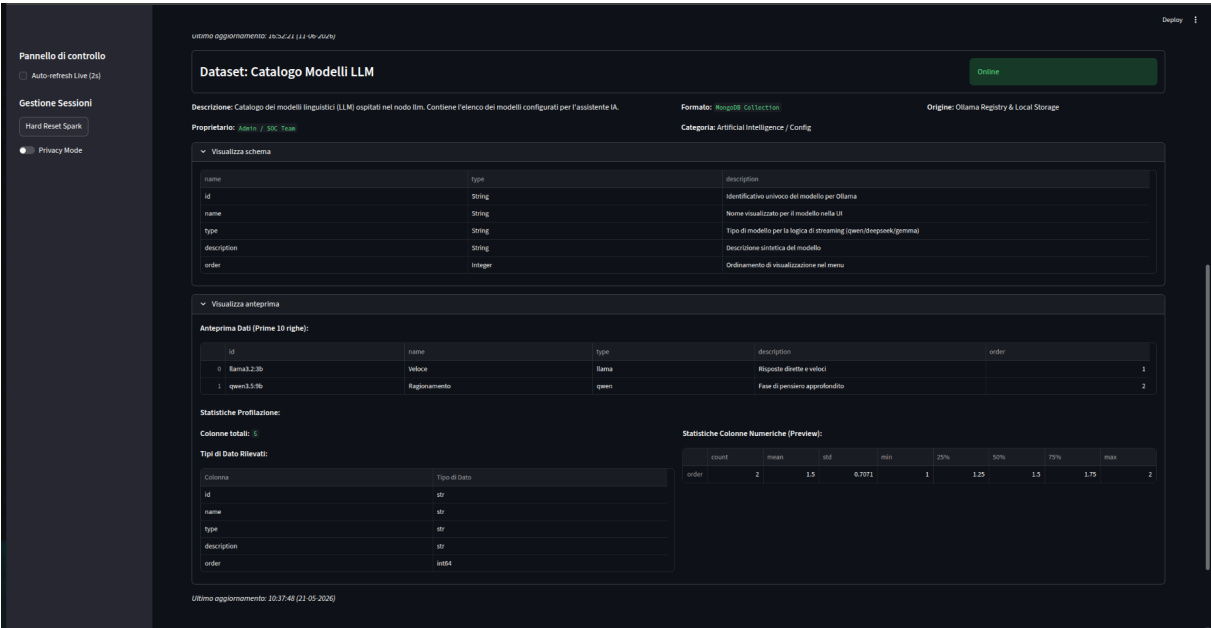


Figura 19: Dashboard che mostra il catalogo degli LLM

5.2 Sniffer

Lo sniffer cattura il traffico di rete in tempo reale tramite tcpdump in un ciclo continuo che scrive blocchi di 1000 pacchetti, evitando che il file cresca indefinitamente. Un ingestore Python rilegge periodicamente questo file con Scapy, estrae i nuovi pacchetti e li scrive su MongoDB in una capped collection, applicando contestualmente le regole di rilevamento/mitigazione descritte in precedenza. Dall’interfaccia è possibile visualizzare gli ultimi pacchetti catturati (con mascheramento opzionale degli IP) e scaricare il file PCAP grezzo per un’analisi approfondita con Wireshark.

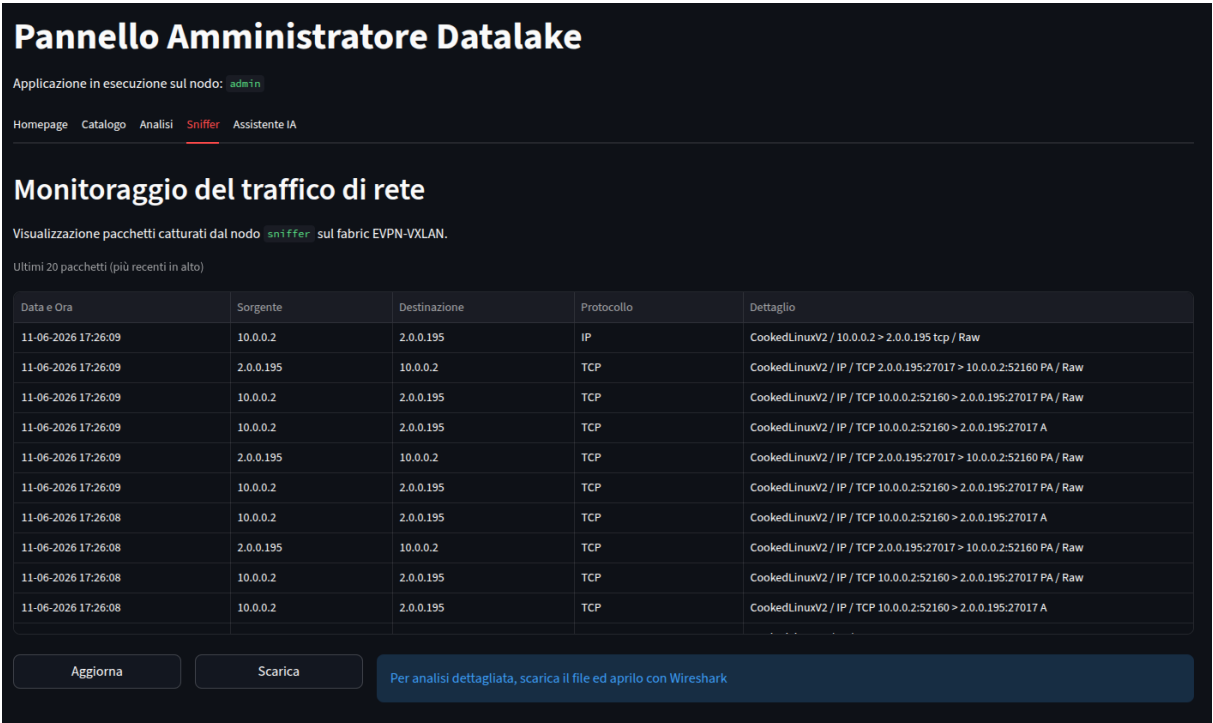


Figura 20: Dashboard che gli ultimi pacchetti catturati dallo sniffer

6 Conclusioni

Il progetto ha dimostrato l’applicazione pratica delle principali tecnologie Big Data in uno scenario realistico:

- ingestion eterogenea: dataset storico Parquet (66M+ record) e dati live PCAP, entrambi interrogabili via Spark SQL grazie alla Data Federation;
- elaborazione distribuita, sfruttando Spark con query SQL complesse su aggregazioni, window functions temporali e join tra sorgenti dati eterogenee;
- visualizzazione interattive, che offrono prospettive complementari sul traffico di rete;
- MongoDB come Data Lake operativa (Capped Collection per stream live, Data Catalog per i metadati);
- integrazione AI: LLM locali con RAG contestuale su dati MongoDB, prompt engineering e generazione automatica di query SQL, senza dipendenza da servizi cloud.

Link repository: https://github.com/andymacale/primo_progetto_big_data